

Appunti di Information Retrieval

Anno Accademico 2026/2027

Indice

1	Introduzione e Architettura dell'Information Retrieval	1
1.1	Definizione e Motivazioni	1
1.1.1	L'Evoluzione nell'Era Generativa: dai Motori Web all'Agentic Search	2
1.1.2	Information Retrieval vs. Data Retrieval (Basi di Dati)	3
1.2	I Due Macro-Obiettivi: Efficacia ed Efficienza	5
1.2.1	Efficienza e Architettura Hardware del Calcolatore . . .	5
1.3	Architettura di un Sistema di Information Retrieval	6
1.3.1	Pipeline di Text Processing	6
1.4	Proprietà del Linguaggio e Modelli di Reperimento	7
1.5	L'Inverted Index (Indice Invertito)	9
1.5.1	Elaborazione delle Query e Algoritmo di Merge	10
1.6	Tecniche di Compressione dell'Indice	12
1.6.1	Binary Interpolative Coding	12
1.6.2	Compressione Basata su Dizionario: LZ77	14
1.7	Learning-to-Rank (LTR) e la Sfida della Non-Differenziabilità	14
1.7.1	Formulazione del Problema di Ranking	14
1.7.2	La Sfida Teorica: La Non-Differenziabilità del Ranking	15
1.7.3	Famiglie di Modelli LTR e Gradienti Surrogati	16
1.8	Evoluzione verso il Neural Information Retrieval	17
1.8.1	Le Tre Famiglie di Rappresentazioni Vettoriali	18
1.9	Ricerca dei Vicini Approssimati (k -ANN) e Scalabilità	19
1.9.1	Forward Index e il Framework TusKANNy	20
2	Valutazione dei Sistemi di Information Retrieval	21
2.1	Come Valutare la Qualità di un Motore di Ricerca	21
2.2	Misurare la Soddisfazione dell'Utente	22
2.3	Il Paradigma di Cranfield e le Collezioni Benchmark	23
2.3.1	La Sfida della Scalabilità Combinatoria	23
2.3.2	Crowdsourcing: Opportunità e Limiti	24
2.3.3	L'Infrastruttura TREC (<i>Text REtrieval Conference</i>) . . .	24

2.3.4	La Tecnica del Pooling	25
2.4	Bisogno Informativo vs. Query	26
2.5	Metriche di Valutazione Non Ordinate (<i>Unranked Measures</i>)	26
2.5.1	Precisione, Richiamo e loro Trade-off	26
2.5.2	La Media Armonica: F-Measure (F_1 -Score)	28
2.5.3	La Misura F_β Pesata	29
2.6	Metriche Basate sul Ranking (<i>Ranked Measures</i> - Rilevanza Binaria)	29
2.6.1	Precision@k ($P@k$) e Recall@k ($R@k$)	29
2.6.2	Average Precision (AP)	30
2.6.3	Mean Average Precision (MAP)	32
2.6.4	Metriche Troncate per il Web e Recommender Systems: AP@k e MAP@k	33
2.6.5	Mean Reciprocal Rank (MRR)	34
2.7	Oltre la Rilevanza Binaria: Rilevanza Graduata e NDCG	35
2.7.1	Cumulative Gain (CG) e Discounted Cumulative Gain (DCG)	36
2.7.2	Normalized Discounted Cumulative Gain (NDCG)	37
2.7.3	Esempio Numerico Completo: Calcolo Passo-Passo	37
2.7.4	Confronto tra Funzioni di Ranking Alternative	38

Capitolo 1

Introduzione e Architettura dell'Information Retrieval

1.1 Definizione e Motivazioni

L'**Information Retrieval** (IR, *Reperimento delle Informazioni*) è la disciplina scientifica e ingegneristica che studia la rappresentazione, la memorizzazione, l'organizzazione e l'accesso a collezioni di dati prevalentemente non strutturati o semi-strutturati, al fine di soddisfare un **bisogno informativo** (*information need*) espresso da un utente [1, 2].

Definizione 1.1.1 (Information Retrieval (Manning et al.)). L'Information Retrieval consiste nel reperire materiale di natura prevalentemente non strutturata (solitamente documenti testuali) che soddisfa un bisogno informativo all'interno di collezioni di grandi dimensioni memorizzate su calcolatori elettronici.

Nell'ecosistema tecnologico contemporaneo, l'Information Retrieval non si limita ai soli motori di ricerca Web generalisti (come Google o Bing), ma costituisce l'infrastruttura abilitante di:

- **E-commerce e marketplace:** reperimento di prodotti basato su descrizioni testuali, attributi e recensioni degli acquirenti (es. Amazon, Alibaba, eBay).
- **Enterprise e Desktop Search:** indicizzazione e ricerca semantica su repository aziendali (documenti, email, codebase sorgente).
- **Sistemi RAG (*Retrieval-Augmented Generation*):** alimentazione di modelli linguistici di grandi dimensioni (LLM) con documenti

esterni contestualmente rilevanti per azzerare le allucinazioni e garantire informazioni aggiornate in tempo reale.

- **Agenti autonomi e motori di raccomandazione:** filtraggio collaborativo e basato su contenuto per la scoperta personalizzata di contenuti e streaming multimediale (es. Netflix, Spotify, Meta, Pinterest).

1.1.1 L'Evoluzione nell'Era Generativa: dai Motori Web all'Agentic Search

L'avvento dei modelli linguistici conversazionali (LLM) non ha reso obsoleto l'Information Retrieval; al contrario, ne ha ampliato radicalmente l'impatto. Un'intervista ideale a un moderno LLM evidenzia come esso stesso operi in stretta sinergia con i principi cardine dell'IR:

1. **Comprensione Semantica della Query:** elaborazione avanzata del linguaggio naturale (NLP) per disambiguare l'intento informativo sottostante.
2. **Matching e Ranking:** selezione e pesatura delle informazioni più rilevanti all'interno di sterminati archivi di conoscenza.
3. **Reperimento Contestuale (*Fresh Information*):** un modello generativo puro è limitato dal momento del suo addestramento (*knowledge cutoff*) e non può rispondere a quesiti su eventi recenti o dati privati senza riaddestrare la rete neurale (operazione economicamente e computazionalmente proibitiva).

Per superare questo limite, i moderni assistenti adottano il paradigma **RAG (*Retrieval-Augmented Generation*)**: il sistema intercetta la richiesta dell'utente, interroga un motore IR via Web o su database documentale vettoriale, inietta i documenti estratti all'interno del contesto dell'LLM e genera una risposta argomentata e supportata da citazioni verificabili.

Agentic Search e il Ciclo Adattivo a Feedback

Mentre il RAG tradizionale adotta una pipeline lineare a singolo passaggio (*Single-hop*: query → retrieval → generation), i problemi complessi richiedono il paradigma emergente della **Agentic Search** (Ricerca Agantica).

Definizione 1.1.2 (Agentic Search). L'**Agentic Search** è un processo di reperimento dinamico in cui un agente intelligente determina in modo autonomo e condizionato quali ricerche successive condurre, basandosi sulle

evidenze parziali raccolte negli stadi precedenti. L'agente ispeziona le fonti, individua discrepanze o lacune conoscitive (*information gaps*), formula nuove query di raffinamento e reitera il ciclo fino alla convergenza informativa.

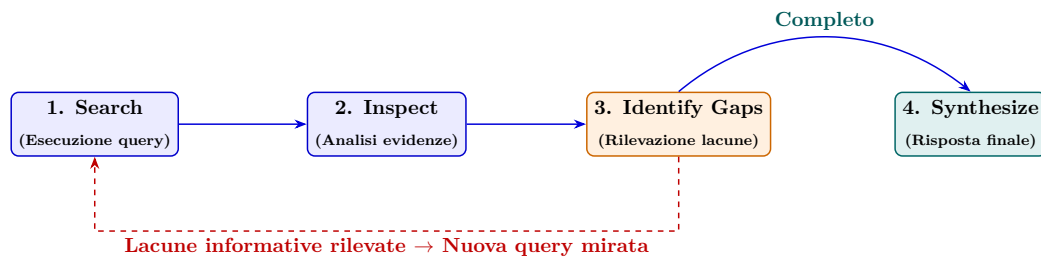


Figura 1.1: Ciclo a retroazione adattivo (*Adaptive Feedback Loop*) dell'Agentic Search.

Si consideri ad esempio il quesito: *"Per quale motivo gli utili della società X sono scesi lo scorso trimestre nonostante l'aumento del fatturato?"*. Un sistema agentic esegue una prima ricerca per reperire il report trimestrale, constata la discrepanza tra ricavi e margini operativi, identifica la necessità di investigare l'andamento dei costi operativi o delle materie prime, formula una seconda query mirata su tali fattori e unisce infine le risultanze in un'argomentazione organica e referenziata.

1.1.2 Information Retrieval vs. Data Retrieval (Basi di Dati)

Una distinzione metodologica basilare risiede nella contrapposizione tra il modello convenzionale dei database relazionali (*Data Retrieval*) e l'Information Retrieval:

La Rilevanza: Il Cardine dell'IR

Mentre in una base di dati un record soddisfa o viola in modo booleano un predicato logico, nell'Information Retrieval il fulcro è la stima del grado di **rilevanza** rispetto al bisogno informativo cognitivo dell'utente. La rilevanza è soggettiva, situata nel contesto operativo ed evolutiva nel tempo.

Dimensione	Data Retrieval (Database SQL)	Information Retrieval
Struttura dei Dati	Dati altamente strutturati con schema relazionale rigido (tabelle, tipi, vincoli di integrità)	Dati non strutturati o semi-strutturati (testo libero, HTML, PDF, metadati)
Linguaggio di Query	Query formali e deterministiche (ad es. SQL) con semantica algebrica esatta	Query in linguaggio naturale o parole chiave (spesso ambigue, brevi e incomplete)
Criterio di Matching	Corrispondenza booleana esatta: un record appartiene o non appartiene al risultato	Corrispondenza parziale e probabilistica: i documenti presentano gradi variabili di rilevanza
Ordinamento Risultati	Nessun ordinamento intrinseco (se non specificato con ORDER BY)	Ordinamento fondamentale basato sulla stima di rilevanza (<i>Relevance Ranking</i>)
Tolleranza all'Errore	Nulla: errori di sintassi o valori inattesi generano errori o insiemi vuoti	Elevata: tolleranza a refusi, sinonimia, polisemia e variabilità morfologica

Tabella 1.1: Confronto analitico tra Data Retrieval tradizionale e Information Retrieval.

1.2 I Due Macro-Obiettivi: Efficacia ed Efficienza

Un sistema di reperimento dell'informazione su larga scala deve simultaneamente perseguire due obiettivi spesso in contrasto:

1. **Efficacia (*Effectiveness*):** Quantifica la qualità intrinseca dei risultati restituiti, misurando quanto accuratamente il ranking soddisfa il bisogno informativo dell'utente (argomento approfondito nel Capitolo 2).
2. **Efficienza (*Efficiency*):** Quantifica il costo computazionale, il throughput (query al secondo) e la latenza di risposta (generalmente richiesta nell'ordine dei 10–50 millisecondi per query su archivi di decine di miliardi di documenti).

1.2.1 Efficienza e Architettura Hardware del Calcolatore

L'efficienza di un motore di ricerca moderno non dipende unicamente dalla complessità asintotica degli algoritmi, ma dall'interazione profonda con l'architettura dei processori e della memoria:

- **Pipelining e Processori Superscalari:** I moderni processori suddividono l'esecuzione delle istruzioni in stadi sequenziali (Instruction Fetch, Decode, Execute, Memory Access, Write Back) ed eseguono molteplici istruzioni per ciclo di clock.
- **Pipeline Hazards (Rischi di Pipeline):**
 - *Structural Hazards:* Conflitti nell'uso simultaneo di risorse hardware limitate (ad es. unità di memoria o registri).
 - *Data Hazards:* Dipendenze logiche tra istruzioni consecutive (RAW: Read-After-Write), che costringono il processore a inserire cicli di stallo (*pipeline bubble*) se i dati non sono pronti.
 - *Control Hazards:* Salto condizionale non previsto. Nella scansione e nell'intersezione delle posting list, un'istruzione condizionale errata (*branch misprediction*) svuota la pipeline, comportando una penalità di 15–20 cicli di clock per ogni errore di predizione.
- **Gerarchia di Memoria (*Memory Hierarchy*):** Il tempo di accesso ai registri della CPU è sub-nanosecondo, alle cache L1/L2/L3 è di pochi cicli, mentre l'accesso alla RAM DDR richiede 50–100 ns e lo storage

secondario (NVMe/SSD) richiede decine di microsecondi. La progettazione delle posting list privilegia layout ad accessi sequenziali contigui per massimizzare il riuso delle linee di cache e sfruttare l'hardware prefetcher.

- **Vettorizzazione SIMD (*Single Instruction Multiple Data*):** L'impiego di registri vettoriali a 128, 256 o 512 bit (AVX-2, AVX-512, ARM NEON) consente di confrontare 8 o 16 identificatori di documento con una singola istruzione macchina, accelerando drasticamente gli algoritmi di intersezione.
- **Calcolo Parallelo e Sharding Distribuito:** Gli indici che superano la capacità di un singolo nodo vengono frammentati (*sharded*), tipicamente per partizionamento documentale (*Document Sharding*), consentendo a cluster di servire migliaia di query parallele aggregando le risposte mediante broker centrali.

1.3 Architettura di un Sistema di Information Retrieval

Come illustrato nello schema di Figura 1.2, un sistema IR adotta una rigorosa separazione tra la **Pipeline Offline** (indicizzazione e addestramento) e la **Pipeline Online** (interrogazione a bassa latenza).

1.3.1 Pipeline di Text Processing

Prima dell'indicizzazione, ogni flusso testuale attraversa quattro fasi sequenziali:

1. **Tokenizzazione (*Tokenization*):** Segmentazione della stringa di caratteri in sequenze di token lessicali, gestendo separatori, acronimi e punteggiatura.
2. **Normalizzazione e Case Folding:** Conversione a caratteri minuscoli e rimozione o armonizzazione di diacritici (es. accenti) per garantire la coerenza tra query e documenti.
3. **Filtraggio delle Stop Words:** Eliminazione di vocaboli ad altissima frequenza ma privi di valore semantico discriminante (articoli, preposizioni, congiunzioni).
4. **Stemming e Lemmatizzazione:**

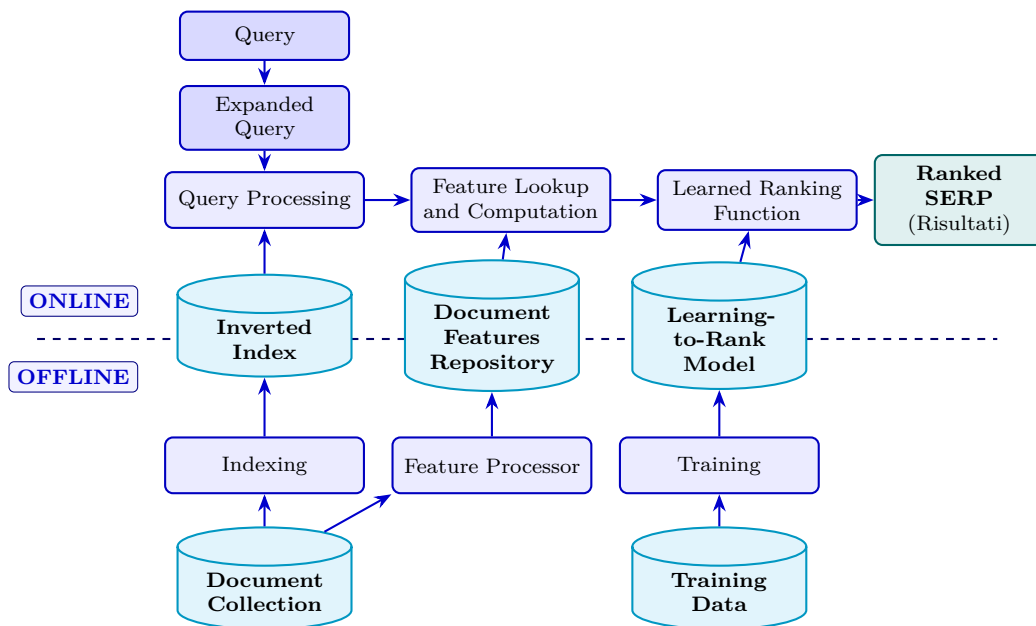


Figura 1.2: Architettura a griglia di un moderno motore di ricerca (pipeline Online vs. Offline).

- *Stemming*: Procedura algoritmica basata su regole euristiche (ad es. l'algoritmo di Porter [1]) che tronca suffissi morfologici per estrarre la radice (*stem*). Esempio: "searching", "searched" → "search".
- *Lemmatizzazione*: Analisi linguistica formale che si avvale di dizionari morfologici per mappare ogni forma flessa al suo lemma canonico. Esempio: forme verbali "sono", "fui" → lemma "essere".

1.4 Proprietà del Linguaggio e Modelli di Reperimento

La distribuzione delle parole nei testi naturali segue leggi empiriche che condizionano sia la progettazione degli indici sia le formule di ranking:

- **Legge di Zipf**: La frequenza $f(r)$ di un termine è inversamente proporzionale al suo rango r nella graduatoria di frequenza:

$$f(r) \propto \frac{1}{r} \quad (1.1)$$

Poche parole frequentissime occupano la maggioranza delle occorrenze, mentre la stragrande maggioranza dei termini compare pochissime volte (*long tail*).

- **Legge di Heaps:** La dimensione del vocabolario $|V|$ cresce come legge di potenza rispetto al numero totale di parole N della collezione:

$$|V| = k \cdot N^\beta \quad (\text{con } 0.4 \leq \beta \leq 0.6) \quad (1.2)$$

Il vocabolario non si satura mai completamente all'aumentare dei documenti.

I principali modelli di reperimento includono:

1. **Modello Booleano:** Match deterministico basato su operatori logici ($\wedge, \vee, \neg$), privo di punteggio intrinseco di rilevanza.
2. **Vector Space Model (TF-IDF):** Rappresentazione di query e documenti come vettori in uno spazio dimensionale $|V|$, pesati con frequenza del termine (TF) e frequenza inversa nei documenti (IDF). La rilevanza è calcolata tramite la similitudine del coseno.
3. **Modello Probabilistico (BM25):** Standard de facto del reperimento lessicale basato sul principio probabilistico del 2-Poisson [2], che formalizza la saturazione asintotica della frequenza del termine e penalizza i documenti eccessivamente lunghi per evitare bias di verbosità:

$$\text{score}_{\text{BM25}}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{tf(t, d) \cdot (k_1 + 1)}{tf(t, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)} \quad (1.3)$$

dove:

- $\text{IDF}(t) = \ln \left(\frac{N - df_t + 0.5}{df_t + 0.5} + 1 \right)$ quantifica la specificità informativa del termine nell'intera collezione di N documenti.
- $tf(t, d)$ è la frequenza grezza del termine t all'interno del documento d .
- $k_1 \geq 0$ (parametro di saturazione del TF, tipicamente calibrato tra 1.2 e 2.0): controlla la rapidità con cui il contributo di occorrenze ripetute dello stesso termine si appiattisce verso un valore asintotico.

- $b \in [0, 1]$ (parametro di penalizzazione della lunghezza, tipicamente $b \approx 0.75$): controlla l'intensità della normalizzazione della lunghezza del documento $|d|$ rapportata alla lunghezza media avgdl dei documenti della collezione. Quando $b = 1$, la penalizzazione per documenti prolissi è massima; per $b = 0$, la lunghezza è ininfluente.

1.5 L'Inverted Index (Indice Invertito)

L'**Inverted Index** inverte l'orientamento logico della memorizzazione documentale, realizzando una mappatura $t \mapsto \mathcal{P}_t$, dove $\mathcal{P}_t$ è la lista ordinata dei documenti contenenti il termine t [1].

Definizione 1.5.1 (Inverted Index). Un Inverted Index è costituito da due componenti:

1. **Dizionario (*Dictionary o Vocabulary*)**: Contiene l'insieme ordinato di tutti i termini unici presenti nella collezione, memorizzato con strutture di lookup rapido (B-Tree o Hash Table), insieme alla frequenza documentale df_t e al puntatore all'inizio della lista.
2. **Posting List**: Per ogni termine t , un array ordinato per identificatore di documento crescente ($\text{DocID}_1 < \text{DocID}_2 < \dots$). Ciascun elemento (posting) può includere metadati accessori quali frequenza locale del termine (tf) e offset posizionali all'interno del documento.

Esempio 1.5.1 (Collezione Giocattolo di Riferimento). Si consideri una collezione di tre documenti didattici:

- d_1 : *"An inverted index is an index storing a mapping from terms to their occurrences in a set of documents."*
- d_2 : *"The inverted index data structure is a central component of a typical search engine indexing algorithm."*
- d_3 : *"The goal is to optimize the speed of the query: find the documents where a set of words occur."*

La Figura 1.3 illustra l'Inverted Index generato per i termini chiave estratti da questa collezione.

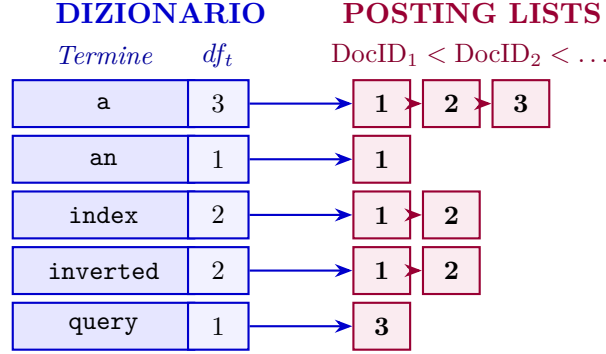


Figura 1.3: Struttura di un Inverted Index per la collezione giocattolo (d_1, d_2, d_3): Dizionario dei termini con frequenza documentale df_t e Posting Lists concatenate.

1.5.1 Elaborazione delle Query e Algoritmo di Merge

L'ordinamento intrinseco dei DocID consente di risolvere le interrogazioni booleane e per frase mediante scansioni coordinate delle posting list:

- **Query Congiuntiva (AND):** Trovare i documenti contenenti contemporaneamente più termini. Ad esempio, per $q = \text{"inverted" AND "index"}$:

$$\mathcal{P}_{\text{inverted}} \cap \mathcal{P}_{\text{index}} = [1, 2] \cap [1, 2] = [1, 2] \implies \{d_1, d_2\}$$

Se invece si richiede $q = \text{"query" AND "inverted" AND "index"}$:

$$(\mathcal{P}_{\text{inverted}} \cap \mathcal{P}_{\text{index}}) \cap \mathcal{P}_{\text{query}} = [1, 2] \cap [3] = \emptyset$$

L'intersezione a due puntatori (Algoritmo 1) garantisce complessità $\mathcal{O}(|P_1| + |P_2|)$. Per ottimizzare query con $k > 2$ termini, la best practice consiste nell'elaborare le liste in ordine crescente di frequenza documentale df_t , partendo dalla posting list più corta per minimizzare le dimensioni intermedie.

- **Query Disgiuntiva (OR):** Trovare documenti contenenti almeno uno dei termini. Per $q = \text{"query" OR "inverted"}$:

$$\mathcal{P}_{\text{query}} \cup \mathcal{P}_{\text{inverted}} = [3] \cup [1, 2] = [1, 2, 3] \implies \{d_1, d_2, d_3\}$$

- **Ricerca per Frase (*Phrase Search*):** Per verificare la presenza di sequenze esatte di parole (es. `"inverted index"`), le posting list memorizzano per ogni DocID gli offset posizionali di comparsa. L'algoritmo verifica che:

$$\text{pos}(\text{index}) - \text{pos}(\text{inverted}) = 1$$

escludendo documenti in cui i termini compaiono in ordine invertito o a distanza arbitraria.

Algorithm 1: Merge Intersection per Query Congiuntive ($P_1 \wedge P_2$)

Dati: Due posting list ordinate P_1 e P_2

Risultato: Lista ordinata R dei documenti presenti in entrambe le liste

```

1  $R \leftarrow \emptyset$ 
2  $i \leftarrow 1; j \leftarrow 1$ 
3 while  $i \leq |P_1|$  e  $j \leq |P_2|$  do
4   if  $P_1[i] == P_2[j]$  then
5      $R.append(P_1[i])$ 
6      $i \leftarrow i + 1$ 
7      $j \leftarrow j + 1$ 
8   else if  $P_1[i] < P_2[j]$  then
9      $i \leftarrow i + 1$ 
10  else
11     $j \leftarrow j + 1$ 
12 return  $R$ 

```

Dalla Teoria alla Pratica: Google Advanced Search

I costrutti teorici dell'algebra booleana e del Phrase Search trovano immediata corrispondenza nell'interfaccia e nella sintassi operativa dei motori di ricerca commerciali (Tabella 1.2).

Operatore Logico IR	Campo Google Advanced Search	Sintassi Operativa ed Esempio
Congiunzione (AND)	<i>all these words</i>	Parole affiancate: <code>tri-colour rat terrier</code>
Ricerca per Frase	<i>this exact word or phrase</i>	Racchiusa tra doppi apici: <code>"rat terrier"</code>
Disgiunzione (OR)	<i>any of these words</i>	Operatore maiuscolo: <code>miniature OR standard</code>
Negazione (NOT)	<i>none of these words</i>	Prefisso meno senza spazi: <code>-rodent -"Jack Russell"</code>
Intervallo Numerico	<i>numbers ranging from ... to</i>	Operatore due punti: <code>10..35 kg, £300..£500</code>

Tabella 1.2: Mappatura diretta tra modelli formali di Query Processing e l'interfaccia di Ricerca Avanzata.

1.6 Tecniche di Compressione dell'Indice

La compressione dell'Inverted Index risponde a una duplice esigenza: ridurre drasticamente l'occupazione di memoria secondaria e RAM, e convertire un collo di bottiglia di trasferimento I/O (lento) in un carico computazionale di decompressione da parte della CPU (estremamente veloce).

1.6.1 Binary Interpolative Coding

Il **Binary Interpolative Coding** (proposto da Moffat e Stuiver [3]) è una tecnica di compressione specificamente concepita per sequenze strettamente crescenti di interi (come i DocID nelle posting list).

Definizione 1.6.1 (Principio di Interpolazione). Data una sottolista ordinata di n interi $L = [x_1, x_2, \dots, x_n]$ nota per essere compresa nell'intervallo $[l, h]$ (ovvero $l \leq x_1 < x_2 < \dots < x_n \leq h$), si seleziona l'elemento centrale $m = \lfloor (n+1)/2 \rfloor$. Poiché devono esistere almeno $m-1$ valori distinti prima di x_m e almeno $n-m$ valori distinti dopo x_m , il valore x_m è strettamente vincolato nell'intervallo ristretto:

$$l + m - 1 \leq x_m \leq h - n + m \quad (1.4)$$

Il numero di possibili valori distinti per x_m è dato da:

$$R = (h - n + m) - (l + m - 1) + 1 = h - l - n + 1 \quad (1.5)$$

Il valore $(x_m - (l + m - 1))$ viene quindi codificato in binario utilizzando esattamente $\lceil \log_2 R \rceil$ bit. Successivamente, l'algoritmo si applica ricorsivamente alla metà sinistra rispetto a $[l, x_m - 1]$ e alla metà destra rispetto a $[x_m + 1, h]$.

Esempio 1.6.1 (Decomposizione Ricorsiva ad Albero). Si consideri la posting list ordinata di $n = 12$ identificatori compresi nell'intervallo $[0, 62]$:

$$L = [3, 4, 7, 13, 14, 15, 21, 25, 36, 38, 54, 62] \quad (n = 12, m = 6, l = 0, h = 62)$$

L'albero di decomposizione (Figura 1.4) partiziona ricorsivamente la sequenza:

- L'elemento centrale $x_6 = 15$ divide la sequenza nelle sottoliste $[3, 4, 7, 13, 14]$ con vincolo $(n = 5, m = 3, l = 0, h = 14)$ e $[21, 25, 36, 38, 54]$ con vincolo $(n = 5, m = 3, l = 16, h = 62)$.
- A livello inferiore, la prima sottolista si divide in $[3, 4]$ con parametri $(n = 2, m = 1, l = 0, h = 6)$ e $[13, 14]$ con parametri $(n = 2, m = 1, l = 8, h = 14)$.
- Le foglie codificano i singoli elementi: ad esempio per $[14]$ con parametri $(n = 1, m = 1, l = 14, h = 14)$, il range di valori ammissibili è:

$$R = 14 - 14 - 1 + 1 = 1 \implies \lceil \log_2 1 \rceil = 0 \text{ bit!}$$

Quando i documenti sono densamente aggregati (*clusterizzati*), il numero di valori ammissibili R crolla verso 1, portando il costo di memorizzazione a zero bit per molti elementi.

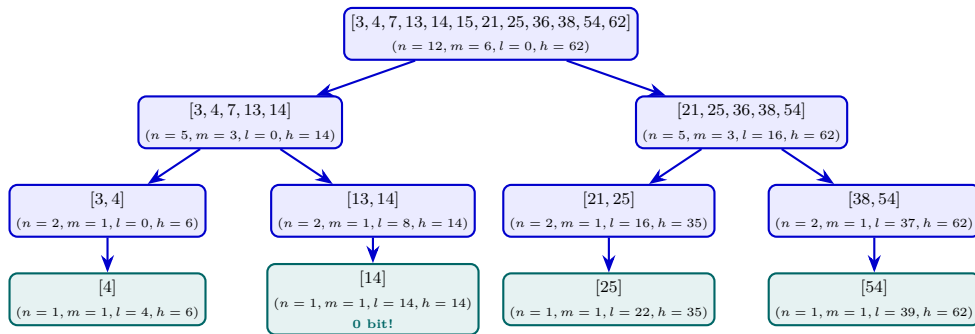


Figura 1.4: Albero di decomposizione dicotomica del Binary Interpolative Coding.

1.6.2 Compressione Basata su Dizionario: LZ77

Per testi generali o stream di token, l'algoritmo **Lempel-Ziv '77** (LZ77) comprime la sequenza tramite una finestra scorrevole divisa in un buffer di ricerca (*Search Buffer*) e un buffer di anteprima (*Lookahead Buffer*), emettendo triple formattate come (offset, length, next_char).

Si consideri la suddivisione analitica della sequenza testuale:

$$T = \underbrace{a}_{(0, 'a')} \underbrace{b}_{(0, 'b')} \underbrace{a \ b \ a}_{(2,3)} \underbrace{a \ a \ a}_{(1,3)} \underbrace{b \ a \ b \ a \ a \ a \ b}_{(7,8)}$$

La successione di puntatori generata è:

$$(0, 'a'), (0, 'b'), (2, 3), (1, 3), (7, 8)$$

In tal modo, lunghe ripetizioni lessicali sono rimpiazzate da compatti offset all'indietro nella finestra di scorrimento. Questa famiglia di algoritmi costituisce la spina dorsale di strumenti ampiamente adottati come **gzip**, **7-Zip** e **LZ4** per l'archiviazione compressa ad alto throughput.

1.7 Learning-to-Rank (LTR) e la Sfida della Non-Differenziabilità

All'interno dell'architettura di un motore di ricerca moderno (cfr. Figura 1.2), i candidati ad alta rilevanza selezionati dall'*Inverted Index* vengono inoltrati alla fase di classificazione e riordino gestita da modelli di Machine Learning (**Learning-to-Rank**, LTR) [4].

1.7.1 Formulazione del Problema di Ranking

Nel paradigma LTR supervisionato, il dataset di addestramento è costituito da un insieme di query, a ciascuna delle quali è associata una lista di documenti valutati da annotatori umani o ricavati da segnali di log:

$$\mathcal{T} = \{(q, \langle (d_1, y_1), (d_2, y_2), \dots, (d_m, y_m) \rangle)\} \quad (1.6)$$

dove $y_i \in \{0, 1, 2, 3, 4\}$ quantifica il grado di rilevanza su scala multi-livello:

- $y = 0$: Irrilevante (*Irrelevant*);
- $y = 1$: Marginalmente rilevante;
- $y = 2$: Rilevante;

- $y = 3$: Molto rilevante;
- $y = 4$: Perfettamente rilevante (*Perfectly relevant*).

A partire da un vettore di feature estratto per ciascuna coppia query-documento $\vec{x}_{q,d} = \Phi(q, d)$, un modello di Machine Learning (spesso assimilato a una complessa funzione "black box") calcola uno score di rilevanza $s_i = f_{\theta}(\vec{x}_{q,d_i})$. La lista ordinata finale (*ranked list*) restituita all'utente viene generata ordinando i documenti per punteggio decrescente:

$$\pi = \text{sort}_{\downarrow}(\{(d_i, s_i)\}_{i=1}^m) \quad (1.7)$$

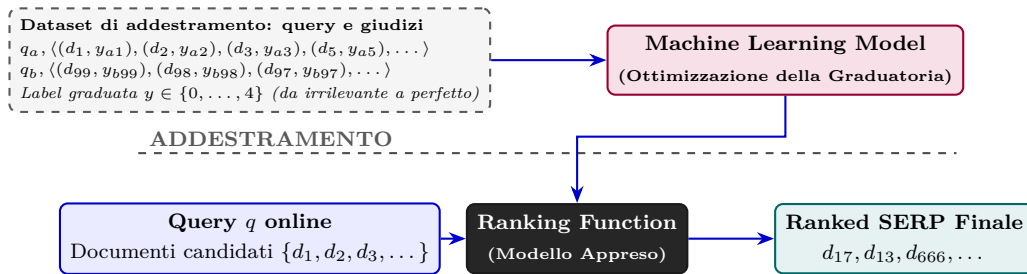


Figura 1.5: Architettura concettuale del Learning-to-Rank (LTR): dall'addestramento con label graduate all'ordinamento della SERP online.

1.7.2 La Sfida Teorica: La Non-Differenziabilità del Ranking

In un problema di regressione convenzionale, l'algoritmo minimizza l'errore quadratico medio $\sum_i (f_{\theta}(x_i) - y_i)^2$. Nell'Information Retrieval, tuttavia, vige un principio cardine:

Il Cardine dell'LTR

The goal is to learn the ranking, not the label: l'obiettivo primario non è stimare accuratamente il valore numerico isolato y , bensì determinare l'ordinamento relativo ottimale dei documenti nella lista presentata all'utente, massimizzando metriche cumulative quali l'*NDCG* (*Normalized Discounted Cumulative Gain*) o la **MAP** (*Mean Average Precision*).

Questa formulazione introduce una difficoltà matematica sostanziale:

- Le metriche di ranking $\mathcal{M}(\pi)$ sono basate sul **rango discreto** (la posizione $1, 2, \dots, k$) assegnato a ciascun documento dalla funzione di ordinamento.
- L'operazione di ordinamento (sort) è una funzione discontinua, a gradini e costante a tratti.
- Variando in modo continuo i parametri θ del modello, le posizioni dei documenti rimangono inalterate (gradiente esattamente nullo) fino al punto in cui due punteggi si scambiano, determinando un salto discontinuo (gradiente non definito):

$$\frac{\partial \mathcal{M}_{\text{NDCG}}(\pi_\theta)}{\partial \theta} = 0 \quad \text{quasi ovunque (a.e.)} \quad (1.8)$$

- Di conseguenza, **non è possibile applicare direttamente la discesa del gradiente stocastico (SGD) o la retropropagazione convenzionale** sulle metriche di ranking ufficiali!

1.7.3 Famiglie di Modelli LTR e Gradienti Surrogati

Per aggirare la non-differenziabilità del ranking, la letteratura ha sviluppato tre metodologie principali:

1. **Approccio Pointwise:** Tratta ciascun documento come un'istanza indipendente, trasformando il problema in una regressione su y o in una classificazione ordinata. Pur consentendo l'uso immediato di SGD, questo approccio ignora la natura intrinsecamente comparativa e competitiva della graduatoria.
2. **Approccio Pairwise (es. RankNet):** Considera coppie di documenti (d_i, d_j) per la stessa query aventi differente rilevanza ($y_i > y_j$). L'obiettivo diviene minimizzare il numero di inversioni, modellando la probabilità che d_i preceda d_j tramite una sigmoide differenziabile:

$$P(d_i \succ d_j) = \frac{1}{1 + e^{-\sigma(s_i - s_j)}}$$

La cross-entropy su queste probabilità è liscia e differenziabile.

3. **Approccio Listwise e Gradienti λ (LambdaMART):** Ottimizza l'intera lista. L'algoritmo **LambdaRank** introduce gradienti virtuali surrogati (λ_{ij}), definiti pesando il gradiente pairwise con la variazione

effettiva della metrica $|\Delta \text{NDCG}_{ij}|$ derivante dallo scambio di posizione tra d_i e d_j :

$$\lambda_{ij} = \frac{-\sigma}{1 + e^{\sigma(s_i - s_j)}} \cdot |\Delta \text{NDCG}_{ij}| \quad (1.9)$$

La combinazione di questi gradienti con alberi di decisione potenziati (*Gradient Boosted Decision Trees*) ha dato origine a **LambdaMART** [4], per oltre un decennio lo standard di riferimento nei motori di ricerca commerciali.

1.8 Evoluzione verso il Neural Information Retrieval

A partire dal 2018, l'avvento dei modelli basati su architettura Transformer e di BERT ha inaugurato l'era del Neural Information Retrieval [5, 6]. I modelli Transformer apprendono la rappresentazione semantica profonda del testo tramite meccanismi di pre-addestramento auto-supervisionato su vasta scala, principalmente il **Masked Language Modeling (MLM)**: durante l'addestramento, una porzione casuale dei token (circa il 15%) viene mascherata tramite il token speciale [MASK], e la rete impara a predirne l'identità lessicale tramite un classificatore denso e una funzione softmax sul vocabolario $|V|$.

Nel reperimento documentale, gli approcci neurali si suddividono in due famiglie architetture fondamentali (Figura 1.6):

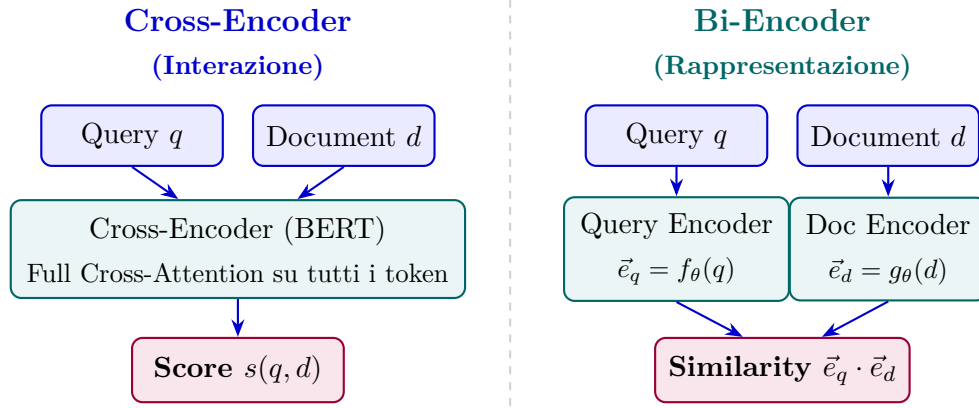


Figura 1.6: Confronto strutturale tra Cross-Encoder (Interazione) e Bi-Encoder (Rappresentazione).

1. **Modelli Basati sull'Interazione (*Cross-Encoders*):** La query e il documento vengono concatenati in un unico input:

$$\text{Input} = [\text{CLS}] \circ q \circ [\text{SEP}] \circ d$$

ed elaborati congiuntamente attraverso tutti i livelli di self-attention del Transformer. Catturano interazioni contestuali profonde tra ciascuna coppia di token, offrendo la massima efficacia predittiva. Tuttavia, il costo computazionale proibitivo ($\mathcal{O}(|\mathcal{D}| \times \text{Transformer})$) ne preclude l'uso su larga scala; vengono impiegati esclusivamente come re-ranker di secondo livello sui top-100 candidati.

2. **Modelli Basati sulla Rappresentazione (*Bi-Encoders*):** La query e il documento vengono proiettati in modo disaccoppiato in uno spazio vettoriale mediante due encoder distinti. I vettori dei documenti possono essere pre-calcolati offline e indicizzati, riducendo il matching online a un prodotto scalare o a una ricerca di vicini approssimati.

1.8.1 Le Tre Famiglie di Rappresentazioni Vettoriali

Nel contesto dei modelli a rappresentazione, si distinguono tre tipologie di embedding:

1. **Dense Single-Vector (es. DPR, Contriever):** Ogni documento e ogni query sono sintetizzati in un unico vettore denso a d dimensioni (ad es. $d = 768$). L'interrogazione richiede indici vettoriali k-ANN su grafi (come HNSW).
2. **Dense Multi-Vector (es. ColBERT [5]):** Viene generato un vettore denso per ciascun token del documento e della query. La rilevanza è determinata tramite il meccanismo di **Late Interaction** con operatore *MaxSim*:

$$S(q, d) = \sum_{t_q \in q} \max_{t_d \in d} (\vec{E}_{t_q} \cdot \vec{E}_{t_d}^T) \quad (1.10)$$

Preserva l'identità dei singoli termini offrendo un'eccellente generalizzazione, a fronte di un costo di memorizzazione più consistente.

3. **Sparse Single-Vector (es. SPLADE [6]):** I documenti e le query sono codificati in vettori sparsi ad altissima dimensione mappati direttamente sui token del vocabolario $|V|$ ($|V| \approx 30\,000$). I coefficienti numerici indicano il peso d'importanza appreso e includono l'espansione automatica di sinonimi.

- **Vantaggi:** Altissima efficacia sia nelle collezioni di dominio (*in-domain*) sia nei test a trasferimento zero-shot (*out-of-domain*).
- **Efficienza:** Più compatti e rapidi rispetto ai modelli multi-vettore, possono essere memorizzati direttamente all'interno delle posting list di un classico Inverted Index.
- **Interpretabilità:** I pesi non nulli corrispondono a parole comprensibili dall'uomo, facilitando il debug e la spiegabilità.

Esempio 1.8.1 (Espansione Semantica di SPLADE). Si consideri la query in linguaggio naturale:

`q = "what shoes do most nba players wear"`

Un modello neurale a vettori sparsi (come SPLADE [6]) attiva un sottoinsieme selezionato di componenti lessicali sul vocabolario $|V|$, producendo un vettore con pesi appresi:

nba: 2.56, shoes: 2.32, shoe: 2.14, basketball: 1.97,
wearing: 1.55, wear: 1.52, most: 1.34, players: 1.25

Questo esempio illustra le due proprietà cardine dei modelli sparsi neurali:

1. **Espansione Semantica dei Termini (Vocabulary Expansion):** Il modello inserisce autonomamente con peso elevato il termine "basketball" (1.97), benché totalmente assente nel testo originario della query, risolvendo il problema del disallineamento lessicale (*vocabulary mismatch*).
2. **Armonizzazione Morfologica:** Attribuisce pesi coordinati a singolari/plurali (shoe / shoes) e forme flesse verbali (wear / wearing), combinando i vantaggi dello stemming linguistico e del deep learning.

1.9 Ricerca dei Vicini Approssimati (k -ANN) e Scalabilità

Nei sistemi diperimento neurale a rappresentazione vettoriale, l'interrogazione online coincide con il problema del **Maximum Inner Product Search (MIPS)**: individuare i primi k documenti della collezione $\mathcal{D}$ che massimizzano il prodotto scalare con il vettore della query $\vec{q}$:

$$\operatorname{argmax}_{v \in \mathcal{D}}^{(k)} \vec{q}^T \cdot \vec{v} \quad (1.11)$$

Nelle collezioni su scala Web moderna (ad es. il benchmark standard MS MARCO):

- La collezione comprende oltre 8.8 milioni di documenti ($|\mathcal{D}| \approx 8.8 \times 10^6$).
- Lo spazio vettoriale lessicale include $|V| \approx 30\,000$ componenti distinte.
- La sparsità è marcata: ciascun documento presenta in media solo circa 150 componenti con coefficiente non nullo.

1.9.1 Forward Index e il Framework TusKANNy

Per evitare il costo computazionale proibitivo della scansione esaustiva lineare, i sistemi k -ANN sparsi memorizzano la collezione mediante una combinazione di indici invertiti e di un **Forward Index** (Indice Diretto). Nel Forward Index, ciascun documento è compresso come una sequenza contigua di coppie (componente, valore):

$$d_i = \langle (c_1, v_1), (c_2, v_2), \dots, (c_m, v_m) \rangle \quad (\text{con } m \approx 150) \quad (1.12)$$

In fase di query ($q = \langle (c_1, v_1), \dots, (c_p, v_p) \rangle$), l'algoritmo calcola il prodotto scalare selettivo solo sulle componenti condivise.

Queste tecniche sono state protagoniste della competizione internazionale **NeurIPS'23 Big-ANN Challenge** e del benchmark **SISAP 2026**. In tale contesto si colloca **TusKANNy** (<https://tuskanny.com/>), una libreria ad altissime prestazioni interamente scritta in Rust sviluppata congiuntamente da ricercatori dell'ISTI-CNR e dell'Università di Pisa. TusKANNy implementa algoritmi avanzati di scansione vettoriale (come *Seismic* e *Kannolo*) capaci di servire oltre 15 000 query al secondo (QPS) con una precisione di richiamo (*recall*) superiore al 90%, operando con latenze sub-millisecondo.

Capitolo 2

Valutazione dei Sistemi di Information Retrieval

2.1 Come Valutare la Qualità di un Motore di Ricerca

La valutazione rigorosa delle prestazioni rappresenta una componente imprescindibile nella progettazione e nell'ottimizzazione dei sistemi di Information Retrieval [1, 2]. L'analisi delle prestazioni investe due dimensioni tra loro complementari:

1. Metriche di Efficienza:

- *Velocità di indicizzazione*: Quanti documenti per ora il sistema è in grado di processare e inserire nell'indice.
- *Indicizzazione incrementale*: Capacità di aggiornare l'indice con nuovi flussi documentali (ad es. +10 000 prodotti al giorno in un e-commerce) senza interruzioni di servizio.
- *Velocità di ricerca (Latenza e Throughput)*: Tempo di risposta (latenza al 95° o 99° percentile) e consumo di CPU/memoria nell'elaborazione di migliaia di query al secondo su collezioni di svariati milioni o miliardi di documenti.

2. Metriche di Efficacia e Qualità dei Servizi Accessori:

- Qualità delle raccomandazioni correlate, accuratezza della correzione ortografica delle query (*spell checking*) e chiarezza dei riassunti testuali visualizzati (*snippet generation*).

- **Efficacia intrinseca della graduatoria:** Misura la capacità fondamentale del motore di ricerca di soddisfare l'utente, ordinando i documenti pertinenti nelle prime posizioni della pagina dei risultati (*Search Engine Results Page*, SERP).

2.2 Misurare la Soddisfazione dell'Utente

Nel contesto operativo dei motori di ricerca commerciali, l'obiettivo ultimo è massimizzare la soddisfazione dell'utente (*User Happiness*). Poiché la felicità è uno stato cognitivo non osservabile direttamente, i sistemi di produzione ricorrono a proxy comportamentali online:

- **Clic e Click-Through Rate (CTR):** Il numero e la proporzione di clic ricevuti dai risultati restituiti.
 - *Criticità:* Titoli sensazionalistici o riassunti ingannevoli (*clickbait*) inducono clic elevati senza che il documento soddisfi realmente l'utente.
 - *La ricerca a zero clic (Zero-Click Search):* L'assenza di clic non è necessariamente un segnale negativo. Quando un utente cerca informazioni fattuali (ad es. il meteo, l'orario di un volo, una conversione di valuta o una definizione enciclopedica), la risposta visualizzata direttamente tramite *Featured Snippet* o *Knowledge Graph* soddisfa istantaneamente il bisogno senza richiedere alcun clic.
- **Conversioni e Transazioni Economiche:** Negli e-commerce, la quota di sessioni di ricerca che sfociano nell'acquisto di un prodotto e l'ammontare monetario speso dall'utente costituiscono metriche primarie di business.
- **Fidelizzazione e Visite Ripetute:** La percentuale di utenti che tornano ad adoperare il motore di ricerca entro un intervallo temporale predefinito (una settimana, un mese).
- **Dwell Time (Tempo di Permanenza):** L'intervallo di tempo che intercorre tra il momento in cui l'utente clicca su un risultato della SERP e l'istante in cui fa ritorno alla pagina dei risultati (*bouncing back*). Un Dwell Time prolungato indica generalmente che il documento ha catturato l'attenzione ed è risultato informativo, mentre un ritorno immediato alla SERP denota insoddisfazione.

La Valutazione nell'Era dei Motori Generativi e LLM

L'emergere di paradigmi basati su modelli linguistici (come ChatGPT, SearchGPT e Perplexity) ridefinisce radicalmente il concetto di Dwell Time e CTR: l'utente non naviga più attraverso link esterni, ma consuma sintesi testuali generate direttamente. In tale scenario, l'accuratezza fattuale, l'assenza di allucinazioni e l'attribuzione corretta delle fonti rimpiazzano il conteggio dei clic tradizionali.

2.3 Il Paradigma di Cranfield e le Collezioni Benchmark

Sebbene i segnali online forniscano un feedback prezioso, essi risultano rumorosi, soggetti a forti bias di posizionamento (*Position Bias*) e non riproducibili sperimentalmente prima del rilascio in produzione. Per consentire valutazioni comparative controllate, la ricerca in IR adotta il **Paradigma di Cranfield**, sviluppato originariamente da Cyril Cleverdon negli anni '60 [7].

Definizione 2.3.1 (Collezione di Test (Test Collection)). Una collezione di test standard per l'Information Retrieval è una tripletta formale $\mathcal{T} = (\mathcal{D}, \mathcal{Q}, \mathcal{J})$ costituita da:

1. Una collezione documentale benchmark $\mathcal{D} = \{d_1, d_2, \dots, d_N\}$;
2. Una suite benchmark di bisogni informativi e query $\mathcal{Q} = \{q_1, q_2, \dots, q_M\}$;
3. Un insieme di giudizi di rilevanza (*Relevance Judgments* o *qrels*) $\mathcal{J} = \{(q, d) \mapsto r_{q,d}\}$, che associa a ciascuna coppia query-documento un valore di rilevanza (binario o graduato).

2.3.1 La Sfida della Scalabilità Combinatoria

La realizzazione manuale esaustiva dei giudizi di rilevanza è materialmente impossibile per archivi realistici. Si consideri uno scenario benchmark con:

$$|\mathcal{D}| = 5 \times 10^6 \text{ documenti}, \quad |\mathcal{Q}| = 50\,000 \text{ query}$$

Lo spazio complessivo delle coppie ammonta a:

$$|\mathcal{D}| \times |\mathcal{Q}| = 5 \times 10^6 \times 5 \times 10^4 = 2.5 \times 10^{11} \text{ coppie query-documento}$$

Ipotizzando che un annotatore umano impieghi in media 2.5 secondi per valutare la rilevanza di un documento:

$$\begin{aligned}\text{Tempo Totale} &= 2.5 \times 10^{11} \times 2.5 \text{ s} = 6.25 \times 10^{11} \text{ secondi} \\ &\approx 173\,611\,111 \text{ ore/persona}\end{aligned}$$

Un impegno titanico che richiederebbe decine di migliaia di anni di lavoro umano ininterrotto! Per ovviare a ciò, i benchmark limitano i giudizi a un sottoinsieme accuratamente campionato di documenti.

2.3.2 Crowdsourcing: Opportunità e Limiti

Una strategia economica consiste nel delegare la valutazione a lavoratori retribuiti su piattaforme online di crowdsourcing (come Amazon Mechanical Turk). Tuttavia, la letteratura scientifica evidenzia come tali giudizi siano affetti da:

- Elevatissima varianza statistica;
- Minore competenza e presenza di spammer;
- Necessità di filtri rigorosi di consenso (ad es. votazione di maggioranza o calcolo dell'accordo inter-annotatore tramite coefficiente κ di Cohen).

2.3.3 L'Infrastruttura TREC (*Text REtrieval Conference*)

Promossa dal National Institute of Standards and Technology (NIST), la conferenza **TREC** rappresenta il punto di riferimento internazionale per la valutazione delle tecnologie di recupero dell'informazione. TREC è strutturata in tracce tematiche (*Tracks*), ciascuna incentrata su specifici casi d'uso (ricerca Web, ricerca clinica, domande-risposte, deep retrieval). Il risultato prodotto da un sistema software che esegue un compito su una collezione di test prende il nome di **Run**.

La Tabella 2.1 documenta l'evoluzione dimensionale delle collezioni benchmark storiche utilizzate nella ricerca in Information Retrieval [1], evidenziando il salto di scala che ha reso indispensabile la nascita di TREC.

Come si evince dai dati, nelle prime collezioni degli anni '60 e '70 (come Cranfield e Medline), le ridotte dimensioni (poche migliaia di record e 1–2 MB) consentivano un'annotazione esaustiva manuale. Con l'avvento di collezioni realistiche da centinaia di migliaia di documenti e gigabyte di testo come TREC, lo spazio combinatorio delle valutazioni è esploso, imponendo l'adozione di metodologie di campionamento selettivo.

Collezione	Documenti (N)	Query	Dimensione (MB)	Termini/Doc	Giudizi (Q-D)
ADI	82	35	—	—	—
AIT	2 109	14	2	400.0	> 10 000
CACM	3 204	64	2	24.5	—
CISI	1 460	112	2	46.5	—
Cranfield	1 400	225	2	53.1	Completi
LISA	5 872	35	3	—	—
Medline	1 033	30	1	—	—
NPL	11 429	93	3	—	—
OSHMED	348 566	106	400	250.0	16 140
Reuters	21 578	672	28	131.0	—
TREC (Tipica)	740 000	200	2 000 (2 GB)	89–3 543	> 100 000

Tabella 2.1: Panoramica comparativa dei principali corpora benchmark storici di valutazione.

2.3.4 La Tecnica del Pooling

Per campionare i documenti da sottoporre ad annotazione umana senza dover valutare l'intera collezione, nei benchmark TREC si adotta la tecnica del **Pooling** (Figura 2.1):

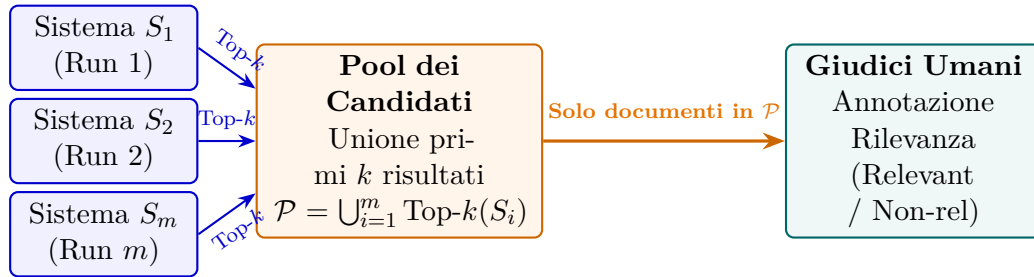


Figura 2.1: Processo di Pooling per la selezione controllata dei documenti da annotare.

1. Si raccolgono le risposte fornite da m sistemi di recupero eterogenei per una determinata query.
2. Si estraggono i primi k documenti restituiti da ciascun sistema (tipicamente $k = 50$ o $k = 100$).
3. Si calcola l'**unione** di tali insiemi eliminando i duplicati, formando il pool $\mathcal{P}$.
4. Gli annotatori umani valutano la rilevanza unicamente per i documenti appartenenti a $\mathcal{P}$.

L'Assunzione Fondamentale del Pooling

Tutti i documenti dell'archivio che non compaiono nel pool (*unpooled*) vengono assunti convenzionalmente come **non rilevanti** ($r = 0$). Sebbene tale assunzione introduca un potenziale bias a sfavore di algoritmi futuri con paradigmi radicalmente differenti, studi empirici condotti da NIST confermano che, a fronte di un pool formato da sistemi sufficientemente diversificati, le conclusioni comparative tra motori di ricerca rimangono stabili e affidabili.

2.4 Bisogno Informativo vs. Query

Una distinzione concettuale decisiva riguarda il divario tra bisogno informativo e formulazione della query:

- **Bisogno Informativo (*Information Need*)**: Rappresenta lo stato cognitivo e l'obiettivo reale dell'utente. Ad esempio: *"Il fondo della mia piscina sta diventando nero e necessita di essere ripulito"*.
- **Query**: L'espressione testuale sintetica digitata nel box di ricerca. Ad esempio: *"pulitore piscina"*.

Proprietà 2.4.1 (Principio di Valutazione della Rilevanza). *La rilevanza di un documento deve essere sempre stimata rispetto all'**Information Need** sottostante, mai rispetto alla mera sequenza di caratteri che compone la query.*

2.5 Metriche di Valutazione Non Ordinate (*Unranked Measures*)

Nel contesto del reperimento non ordinato (come nel modello booleano classico), il sistema restituisce un insieme non graduato di documenti. La valutazione si fonda sulla suddivisione dell'universo documentale mediante la matrice di contingenza (Tabella 2.2):

2.5.1 Precisione, Richiamo e loro Trade-off

Definizione 2.5.1 (Precisione (*Precision*, P)). La frazione dei documenti restituiti dal sistema che risultano effettivamente rilevanti:

$$P = \frac{TP}{TP + FP} = P(\text{rilevante} \mid \text{reperito}) \quad (2.1)$$

	Documento Rilevante	Documento Non Rilevante
Documento Reperito	True Positive (TP)	False Positive (FP)
Documento Non Reperito	False Negative (FN)	True Negative (TN)

Tabella 2.2: Matrice di contingenza per la classificazione binaria query-documento.

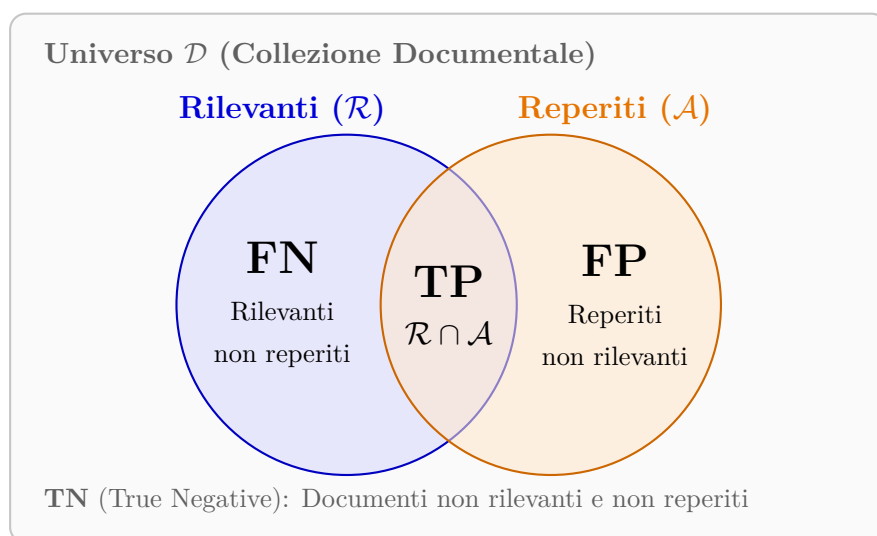


Figura 2.2: Rappresentazione insiemistica del reperimento documentale e delle regioni della matrice di confusione.

Definizione 2.5.2 (Richiamo (*Recall*, R)). La frazione dei documenti rilevanti complessivamente presenti nella collezione che il sistema è riuscito a reperire:

$$R = \frac{TP}{TP + FN} = P(\text{reperito} \mid \text{rilevante}) \quad (2.2)$$

Esiste un intrinseco **trade-off** tra Precisione e Richiamo:

- Un sistema che restituisce l'intera collezione documentale (Retrieved = $\mathcal{D}$) ottiene banamente un richiamo perfetto $R = 100\%$, ma la sua precisione crolla a livelli minimi ($P \approx 0$).
- Viceversa, un sistema conservativo che restituisce unicamente il singolo documento più sicuro può conseguire $P = 100\%$, pagando il prezzo di un richiamo trascurabile.

2.5.2 La Media Armonica: F-Measure (F_1 -Score)

Per sintetizzare Precisione e Richiamo in un unico valore scalare che premi l'equilibrio tra le due grandezze, si adotta la **F-Measure** (o F_1 -score), definita come la **media armonica** di P ed R :

$$F_1 = \frac{2}{\frac{1}{P} + \frac{1}{R}} = \frac{2 \cdot P \cdot R}{P + R} = \frac{2 \cdot TP}{2 \cdot TP + FP + FN} \quad (2.3)$$

Proprietà 2.5.1 (Proprietà della Media Armonica). *La media armonica gode di proprietà matematiche fondamentali per la valutazione:*

1. È sempre minore o uguale alla media geometrica e alla media aritmetica:

$$\frac{2}{\frac{1}{a} + \frac{1}{b}} \leq \sqrt{a \cdot b} \leq \frac{a + b}{2}$$

2. Quando i due argomenti differiscono notevolmente, la media armonica collassa verso il **valore minimo** anziché posizionarsi a metà strada.

Esempio 2.5.1 (Comportamento Limite dell' F -Measure). Si consideri un sistema banale che restituisce l'intera collezione: $R = 1.0$ e $P = 0.0001$.

- La media aritmetica assegna un punteggio fuorviante pari a:

$$\frac{1.0 + 0.0001}{2} \approx 0.50 \quad (50\%)$$

- La media armonica (F_1) demolisce giustamente la valutazione:

$$F_1 = \frac{2 \cdot 1.0 \cdot 0.0001}{1.0 + 0.0001} \approx 0.0002 \quad (0.02\%)$$

Ciò dimostra per quale motivo la media aritmetica sia inadeguata e l' F -measure costituisca la scelta corretta.

2.5.3 La Misura F_β Pesata

Qualora si desideri attribuire pesi differenziati a Precisione e Richiamo, si introduce un parametro di ponderazione $\alpha \in [0, 1]$:

$$F = \frac{1}{\alpha \frac{1}{P} + (1 - \alpha) \frac{1}{R}} \quad (2.4)$$

Esprimendo α in funzione di un parametro β tale che $\alpha = \frac{1}{1+\beta^2}$, si ricava la formulazione classica di F_β :

$$F_\beta = \frac{(1 + \beta^2) \cdot P \cdot R}{\beta^2 \cdot P + R} \quad (2.5)$$

- Valori di $\beta < 1$ (ad es. $\beta = 0.5$) enfatizzano la **Precisione** rispetto al Richiamo (utile nelle ricerche su Web generico).
- Valori di $\beta > 1$ (ad es. $\beta = 2.0$) enfatizzano il **Richiamo** rispetto alla Precisione (fondamentale nelle ricerche brevettuali, legali e mediche, dove omettere un documento rilevante comporta danni gravi).

2.6 Metriche Basate sul Ranking (*Ranked Measures* - Rilevanza Binaria)

Nei motori di ricerca moderni i documenti non vengono restituiti come insiemi informi, ma ordinati in base a uno score di rilevanza decrescente. Inoltre, gli utenti Web raramente oltrepassano la prima pagina dei risultati (i primi 10 o 20 elementi).

2.6.1 Precision@k ($P@k$) e Recall@k ($R@k$)

Definizione 2.6.1 (Precision@k). Fissata una soglia di rango $k \in \mathbb{N}^+$, la Precision@k corrisponde alla proporzione di documenti rilevanti presenti nei primi k posti della graduatoria:

$$P@k = \frac{1}{k} \sum_{i=1}^k \text{rel}_i \quad (2.6)$$

dove $\text{rel}_i \in \{0, 1\}$ denota la rilevanza binaria del documento al rango i .

Esempio 2.6.1 (Calcolo di Precision@k). Si consideri una lista ordinata in cui la sequenza di rilevanza binaria dei primi 5 documenti è data da $\langle 1, 1, 0, 0, 1 \rangle$:

- $P@1 = \frac{1}{1} = 1.00$
- $P@2 = \frac{2}{2} = 1.00$
- $P@3 = \frac{2}{3} \approx 0.66$
- $P@4 = \frac{2}{4} = 0.50$ (si noti che $P@4 < P@3$)
- $P@5 = \frac{2}{5} = 0.40$

La Distorsione di $P@k$ nella Media tra Query

Sebbene $P@k$ sia intuitivo per stimare la qualità della prima pagina, presenta un difetto metodologico critico quando viene mediato su insiemi di query eterogenee:

- Se per una query specifica q_A esistono solo $R_A = 2$ documenti rilevanti nell'intera collezione, il valore massimo teoricamente raggiungibile per $P@10(q_A)$ è pari a $\frac{2}{10} = 0.20$, anche qualora il motore collochi perfettamente entrambi i documenti ai primi due ranghi!
- Calcolando la media aritmetica di $P@10$ tra query, il motore viene ingiustamente penalizzato dalla rarità intrinseca dei documenti rilevanti di alcune query, e non da un errore dell'algoritmo di ranking.

In modo analogo, la metrica complementare $Recall@k$ ($R@k$) calcola la frazione dei documenti rilevanti complessivi reperiti entro il rango k :

$$Recall@k = \frac{1}{R} \sum_{i=1}^k rel_i \quad (2.7)$$

2.6.2 Average Precision (AP)

L'**Average Precision** (AP) sintetizza l'intera curva Precision-Recall in un singolo indicatore scalare per una specifica query, valutando la precisione istantanea in corrispondenza di ogni posizione in cui il richiamo aumenta.

Definizione 2.6.2 (Average Precision). Dato un ordinamento di N documenti per una query con R documenti rilevanti totali presenti nella collezione, l'AP è definita come:

$$AP = \frac{1}{R} \sum_{k=1}^N P@k \cdot rel_k \quad (2.8)$$

Proprietà 2.6.1 (Interpretazione Geometrica e Ruolo di $1/R$). *L'Average Precision possiede una chiara interpretazione analitica:*

1. Ogni volta che si incontra un documento rilevante al rango k , il richiamo compie un gradino discreto di ampiezza $\Delta\text{Recall} = \frac{1}{R}$. L'AP calcola la somma delle aree dei rettangoli $P@k \cdot \Delta\text{Recall}$, rappresentando l'**approssimazione numerica dell'Area Sotto la Curva Precision-Recall (AUC-PR)**.
2. La normalizzazione per R (il numero totale di documenti rilevanti nella collezione) garantisce che i documenti rilevanti non recuperati entro i primi N ranghi contribuiscano implicitamente con precisione pari a 0, penalizzando severamente i sistemi affetti da scarso richiamo.

	Ordinamento del Motore di Ricerca 				
Posizione (Rango):	$k = 1$	$k = 2$	$k = 3$	$k = 4$	$k = 5$
Documento:	Rel	NonRel	Rel	NonRel	Rel
Precision@k:	1.00	—	0.67	—	0.60

Figura 2.3: Rappresentazione schematica del calcolo di Average Precision su 5 documenti con $R = 3$ rilevanti totali.

Esempio 2.6.2 (Calcolo di Base con $R = 3$ Rilevanti). Si consideri lo scenario di Figura 2.3 con $R = 3$ documenti rilevanti totali, posizionati ai ranghi $k_1 = 1$, $k_2 = 3$ e $k_3 = 5$:

$$P@1 = \frac{1}{1} = 1.00, \quad P@3 = \frac{2}{3} \approx 0.667, \quad P@5 = \frac{3}{5} = 0.60$$

L'Average Precision risulta:

$$AP = \frac{1}{3} \left(1.00 + \frac{2}{3} + \frac{3}{5} \right) = \frac{1}{3} (2.267) \approx 0.756 \quad (75.6\%) \quad (2.9)$$

Esempio 2.6.3 (Confronto Comparativo tra Due Ordinamenti). Si consideri una query per la quale esistono $R = 6$ documenti rilevanti nella collezione. Due motori di ricerca alternativi restituiscono i seguenti primi 10 documenti:

- **Ranking #1:** Rilevanti concentrati nelle prime posizioni:

⟨Rel, NonRel, Rel, Rel, Rel, Rel, NonRel, NonRel, NonRel, Rel⟩

Rango (k)	Ranking #1			Ranking #2		
	Esito	Recall	Precision	Esito	Recall	Precision
1	Rel	0.17	1.00	NonRel	0.00	—
2	NonRel	0.17	—	Rel	0.17	0.50
3	Rel	0.33	0.67	NonRel	0.17	—
4	Rel	0.50	0.75	NonRel	0.17	—
5	Rel	0.67	0.80	Rel	0.33	0.40
6	Rel	0.83	0.83	Rel	0.50	0.50
7	NonRel	0.83	—	Rel	0.67	0.57
8	NonRel	0.83	—	NonRel	0.67	—
9	NonRel	0.83	—	Rel	0.83	0.56
10	Rel	1.00	0.60	Rel	1.00	0.60
Average Precision (AP)			0.78	0.52		

Tabella 2.3: Calcolo comparativo dettagliato di Average Precision tra Ranking #1 e Ranking #2.

- **Ranking #2:** Rilevanti dispersi e ritardati:

⟨NonRel, Rel, NonRel, NonRel, Rel, Rel, Rel, NonRel, Rel, Rel⟩

La Tabella 2.3 traccia l'evoluzione rango per rango di Precisione e Richiamo per entrambi i sistemi.

I rispettivi valori di AP risultano:

$$AP(\text{Ranking \#1}) = \frac{1.00 + 0.67 + 0.75 + 0.80 + 0.83 + 0.60}{6} = \frac{4.65}{6} \approx \mathbf{0.78}$$

$$AP(\text{Ranking \#2}) = \frac{0.50 + 0.40 + 0.50 + 0.57 + 0.56 + 0.60}{6} = \frac{3.13}{6} \approx \mathbf{0.52}$$

Sebbene entrambi i motori raggiungano un richiamo finale identico ($R = 100\%$ sui 10 documenti), Ranking #1 ottiene un punteggio nettamente superiore (0.78 vs 0.52) poiché premia la densità di documenti rilevanti nella parte alta della graduatoria.

2.6.3 Mean Average Precision (MAP)

Quando si valuta un motore di ricerca su un banco di prova comprendente $|Q|$ bisogni informativi distinti, si calcola la **Mean Average Precision** (MAP) come media aritmetica dei valori di AP:

Definizione 2.6.3 (Mean Average Precision).

$$MAP = \frac{1}{|Q|} \sum_{q \in Q} AP(q) \quad (2.10)$$

Proprietà 2.6.2 (Macro-Averaging vs. Micro-Averaging). *Il MAP adotta formalmente una procedura di **Macro-Averaging**: ciascuna query possiede un peso identico pari a $1/|\mathcal{Q}|$.*

- *Se si adottasse il Micro-Averaging (ovvero sommare tutte le precisioni istantanee e dividere per il numero totale di documenti rilevanti dell'intera collezione), la misura verrebbe totalmente monopolizzata da pochissime query iper-generiche (ad es. "film" o "notizie", aventi decine di migliaia di documenti rilevanti), mascherando il fallimento del motore sulle query rare e specifiche (*long-tail*).*
- *Il Macro-Averaging garantisce che ogni singola esigenza informativa dell'utente riceva pari dignità nella valutazione finale.*

Esempio 2.6.4 (Calcolo di MAP su Due Query). Si consideri la valutazione congiunta di due query q_1 e q_2 :

- **Query 1 ($R_1 = 5$ rilevanti totali):** Il sistema restituisce 5 rilevanti ai ranghi $k \in \{1, 3, 6, 9, 10\}$ con precisioni istantanee pari a 1.00, 0.67, 0.50, 0.44 e 0.50:

$$AP(q_1) = \frac{1.00 + 0.67 + 0.50 + 0.44 + 0.50}{5} = \frac{3.11}{5} \approx \mathbf{0.62}$$

- **Query 2 ($R_2 = 3$ rilevanti totali):** Il sistema restituisce i 3 rilevanti ai ranghi $k \in \{2, 5, 7\}$ con precisioni istantanee pari a 0.50, 0.40 e 0.43:

$$AP(q_2) = \frac{0.50 + 0.40 + 0.43}{3} = \frac{1.33}{3} \approx \mathbf{0.44}$$

Il valore di Mean Average Precision del motore sui due compiti ammonta a:

$$MAP = \frac{AP(q_1) + AP(q_2)}{2} = \frac{0.62 + 0.44}{2} = \mathbf{0.53} \quad (2.11)$$

2.6.4 Metriche Troncate per il Web e Recommender Systems: AP@k e MAP@k

Nei moderni motori di ricerca su Web e nei sistemi di raccomandazione, calcolare l'AP sull'intera collezione risulta impraticabile (perché il numero totale R di rilevanti non è noto a priori) e poco rilevante per l'esperienza utente. Si adotta pertanto la variante troncata a soglia fissa K (**MAP@K**):

Definizione 2.6.4 (AP@K e MAP@K). Fissata la soglia di taglio K (tipicamente $K = 10$ o $K = 20$), l’Average Precision at K per la query q_i è definita come:

$$AP@K(q_i) = \frac{1}{K_i} \sum_{k=1}^K P@k(q_i) \cdot \text{rel}(q_i, k) \quad (2.12)$$

dove il fattore di normalizzazione K_i è dato dal numero di documenti rilevanti presenti nei primi K posti:

$$K_i = \sum_{k=1}^K \text{rel}(q_i, k) \quad (\text{oppure } K_i = \min(R_i, K)) \quad (2.13)$$

La metrica globale **MAP@K** su un insieme di query $\mathcal{Q}$ corrisponde alla media:

$$MAP@K(\mathcal{Q}) = \frac{1}{|\mathcal{Q}|} \sum_{q_i \in \mathcal{Q}} AP@K(q_i) \quad (2.14)$$

Questa formulazione risolve brillantemente il limite di $P@K$: normalizzando per K_i anziché rigidamente per K , una query con soli 2 rilevanti che li colloca ai primi 2 posti ottiene $AP@10 = 1.0$ anziché essere artificialmente limitata a 0.20.

2.6.5 Mean Reciprocal Rank (MRR)

In scenari quali le ricerche di elementi noti (*Known-Item Search*), le query navigazionali (ad es. cercare l’home page di un ateneo) o il reperimento di risposte fattuali, l’utente è interessato unicamente a individuare la **prima risposta corretta**. Lo sforzo cognitivo e il tempo speso dall’utente sono inversamente proporzionali alla posizione occupata dal primo risultato rilevante:

Definizione 2.6.5 (Reciprocal Rank e MRR). Sia $\text{rank}_1(q)$ la posizione del primo documento rilevante restituito per la query q . Il **Reciprocal Rank** (RR) è dato da:

$$RR(q) = \frac{1}{\text{rank}_1(q)} \quad (2.15)$$

Il **Mean Reciprocal Rank** (MRR) è la media aritmetica su tutte le query:

$$MRR = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \frac{1}{\text{rank}_1(q)} \quad (2.16)$$

Se per una query il primo documento rilevante si trova al rango 1, il punteggio è 1.0; se scende al rango 2 diviene 0.5; se scende al rango 10 crolla a 0.1.

2.7 Oltre la Rilevanza Binaria: Rilevanza Graduata e NDCG

Nelle applicazioni Web reali, l'ipotesi che un documento sia puramente rilevante (1) o non rilevante (0) risulta insufficiente. Si consideri la seguente query reale:

Query: "information retrieval unipi"

Una tipica SERP commerciale restituisce risultati con livelli qualitativi marcatamente differenti:

1. **DidaWiki del Corso (Docente, CFU, Programma, Materiale didattico):** Rappresenta la pagina principale che soddisfa pienamente l'intento dell'utente $\implies$ **Altamente Rilevante** ($r = 3$).
2. **Portale Ufficiale UniPi Valutami (Scheda d'esame e requisiti istituzionali):** Pagina autorevole ma meno ricca di dispense didattiche $\implies$ **Rilevante** ($r = 2$).
3. **Brochure PDF del Corso di Laurea Magistrale:** Contiene solo una breve menzione del corso $\implies$ **Marginalmente Rilevante** ($r = 1$).
4. **Articoli di cronaca o pagine non correlate:** $\implies$ **Irrilevante** ($r = 0$).

Tutti i primi tre documenti sono in qualche misura attinenti, ma un utente sarà soddisfatto solo se la pagina più informativa compare in cima. Per catturare questa granularità, la letteratura adotta scale a più livelli (tipicamente $r \in \{0, 1, 2, 3\}$ o $\{0, \dots, 4\}$).

L'approccio del **Discounted Cumulative Gain** (DCG), introdotto da Järvelin e Kekäläinen [8], poggia su due assunzioni fondamentali:

1. I documenti altamente rilevanti apportano un'utilità (*gain*) di gran lunga superiore rispetto a documenti solo marginalmente rilevanti.
2. Minore è la posizione ordinale (rango) di un documento nella graduatoria, minore è la probabilità che l'utente lo esamini; pertanto, il guadagno deve essere progressivamente scontato (*discounted*) verso il basso.

2.7.1 Cumulative Gain (CG) e Discounted Cumulative Gain (DCG)

Definizione 2.7.1 (Cumulative Gain a Rango p). Dato il vettore delle rilevanze graduate $\mathbf{r} = [r_1, r_2, \dots, r_p]$, il Cumulative Gain è la somma lineare dei guadagni:

$$CG_p = \sum_{i=1}^p r_i \quad (2.17)$$

Il CG_p è insensibile alla posizione: scambiare un documento altamente rilevante al rango 1 con uno al rango 10 non altera il valore di CG_p .

Definizione 2.7.2 (Discounted Cumulative Gain (Formulazione Standard)). Il DCG applica uno sconto logaritmico basato sul rango della posizione i :

$$DCG_p = r_1 + \sum_{i=2}^p \frac{r_i}{\log_2(i)} \quad (2.18)$$

Significato Comportamentale dello Sconto Logaritmico

Lo sconto $\frac{1}{\log_2(i)}$ riflette il decadimento esponenziale dell'attenzione visiva dell'utente secondo il modello di lettura a cascata:

- Al rango $i = 4$, il fattore di sconto è $\frac{1}{\log_2(4)} = \frac{1}{2}$: il valore informativo percepito si dimezza rispetto alla prima posizione.
- Al rango $i = 8$, il fattore è $\frac{1}{\log_2(8)} = \frac{1}{3}$.
- Al rango $i = 16$, il fattore scende a $\frac{1}{\log_2(16)} = \frac{1}{4}$.

Nei motori di ricerca commerciali (Google, Bing) si impiega frequentemente la **formulazione esponenziale**:

$$DCG_p^{\text{exp}} = \sum_{i=1}^p \frac{2^{r_i} - 1}{\log_2(i + 1)} \quad (2.19)$$

Chiarimento sulla Notazione degli Indici

Si noti la differenza nell'argomento del logaritmo:

- Nell'Equazione (2.18) il primo termine r_1 viene estratto per evitare $\log_2(1) = 0$ al denominatore, mentre la sommatoria parte da $i = 2$ con $\log_2(i)$.
- Nell'Equazione (2.19) la sommatoria include direttamente il rango $i = 1$ adottando $\log_2(i + 1)$ al denominatore ($\log_2(1 + 1) = \log_2(2) = 1$).
- Inoltre, il numeratore $2^{r_i} - 1$ enfatizza in modo non lineare i documenti di massima rilevanza ($r = 3 \implies 2^3 - 1 = 7$, mentre $r = 1 \implies 2^1 - 1 = 1$), penalizzando aspramente il loro mancato posizionamento nei primi ranghi.

2.7.2 Normalized Discounted Cumulative Gain (NDCG)

Poiché query differenti presentano un numero variabile di documenti rilevanti, il valore grezzo di DCG_p non è confrontabile tra query diverse. Si ricorre quindi all'**Ideal DCG** ($IDCG_p$), ottenuto calcolando il DCG della graduatoria ideale che ordina tutti i documenti per rilevanza decrescente:

$$NDCG_p = \frac{DCG_p}{IDCG_p} \in [0, 1] \quad (2.20)$$

2.7.3 Esempio Numerico Completo: Calcolo Passo-Passo

Si consideri la seguente graduatoria di 10 documenti, valutati su scala 0–3:

$$\mathbf{r} = [3, 2, 3, 0, 0, 1, 2, 2, 3, 0]$$

La sequenza ideale ottenuta ordinando per rilevanza non crescente è:

$$\mathbf{r}^* = [3, 3, 3, 2, 2, 2, 1, 0, 0, 0]$$

La Tabella 2.4 riporta l'evoluzione dettagliata di ogni singolo termine posizionale.

i	r_i	Sconto	Guadagno Scontato	$DCG@i$	r_i^*	Guadagno Ideale	$IDCG@i$	$NDCG@i$
1	3	1.000	3.00	3.00	3	3.00	3.00	1.00
2	2	$\log_2(2) = 1.000$	2.00	5.00	3	3.00	6.00	0.83
3	3	$\log_2(3) = 1.585$	1.89	6.89	3	1.89	7.89	0.87
4	0	$\log_2(4) = 2.000$	0.00	6.89	2	1.00	8.89	0.76
5	0	$\log_2(5) = 2.322$	0.00	6.89	2	0.86	9.75	0.71
6	1	$\log_2(6) = 2.585$	0.39	7.28	2	0.77	10.52	0.69
7	2	$\log_2(7) = 2.807$	0.71	7.99	1	0.36	10.88	0.73
8	2	$\log_2(8) = 3.000$	0.67	8.66	0	0.00	10.88	0.80
9	3	$\log_2(9) = 3.170$	0.95	9.61	0	0.00	10.88	0.88
10	0	$\log_2(10) = 3.322$	0.00	9.61	0	0.00	10.88	0.88

Tabella 2.4: Tracciamento dettagliato del calcolo di $DCG@i$, $IDCG@i$ e $NDCG@i$ per $i = 1, \dots, 10$.

2.7.4 Confronto tra Funzioni di Ranking Alternative

Si consideri il caso studio di 4 documenti $\{d_1, d_2, d_3, d_4\}$ con gradi di rilevanza:

$$r(d_1) = 0, \quad r(d_2) = 1, \quad r(d_3) = 2, \quad r(d_4) = 2$$

- **Ground Truth (GT):** Ordinamento $\langle d_4, d_3, d_2, d_1 \rangle$ con vettori di rilevanza $[2, 2, 1, 0]$:

$$IDCG_4 = 2 + \frac{2}{\log_2(2)} + \frac{1}{\log_2(3)} + 0 = 2 + 2 + 0.6309 = 4.6309$$

$$NDCG_{GT} = 1.00$$

- **Ranking Function 1 (RF_1):** Ordinamento $\langle d_3, d_4, d_2, d_1 \rangle$ con rilevanze $[2, 2, 1, 0]$. Poiché i primi due documenti possiedono il medesimo grado ($r = 2$), l'inversione non produce alcuna penalità posizionale:

$$DCG_{RF_1} = 2 + \frac{2}{\log_2(2)} + \frac{1}{\log_2(3)} = 4.6309$$

$$NDCG_{RF_1} = \frac{4.6309}{4.6309} = 1.00$$

- **Ranking Function 2 (RF_2):** Ordinamento $\langle d_3, d_2, d_4, d_1 \rangle$ con rilevanze $[2, 1, 2, 0]$. In questo caso, il documento d_2 ($r = 1$) scavalca erroneamente d_4 ($r = 2$):

$$DCG_{RF_2} = 2 + \frac{1}{\log_2(2)} + \frac{2}{\log_2(3)} + 0 = 2 + 1 + \frac{2}{1.585} = 4.2619$$

$$NDCG_{RF_2} = \frac{DCG_{RF_2}}{IDCG_4} = \frac{4.2619}{4.6309} \approx 0.9203$$

Questo calcolo dimostra con precisione matematica come la metrica NDCG penalizzi le inversioni di rango proporzionalmente alla gravità dello scambio di rilevanza.

Bibliografia

- [1] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [2] Stefan Büttcher, Charles L. A. Clarke, and Gordon V. Cormack. *Information Retrieval: Implementing and Evaluating Search Engines*. MIT Press, 2010.
- [3] Alistair Moffat and Luke Stuiver. Binary interpolative coding for effective index compression. *Information Processing & Management*, 36(5):667–687, 2000.
- [4] Christopher J. C. Burges. From RankNet to LambdaRank to LambdaMART: An overview. Technical Report MSR-TR-2010-82, Microsoft Research, 2010.
- [5] Omar Khattab and Matei Zaharia. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 39–48, 2020.
- [6] Thibault Formal, Benjamin Piwowarski, and Stéphane Clinchant. SPLADE: Sparse lexical and expansion model for information retrieval. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2288–2292, 2021.
- [7] Cyril W. Cleverdon. The Cranfield tests on index language devices. *Aslib Proceedings*, 19(6):173–194, 1967.
- [8] Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of retrieval techniques. *ACM Transactions on Information Systems (TOIS)*, 20(4):422–446, 2002.